

WEEK 1- PART2

PYTHON

Review??

ReCheck



INSTALLING ENVIRONMENT

- Anaconda
- Collaboratory
 - Title
 - Code
 - Section



BASIC COMMAND

- **print**

- To print, try `print (7+5)`

- **#**

- In line comment (similar to `//` in C)



CALCULATION

■ BASIC CALCULATION

■ +, -, *, /

e.g. 5+6

■ ** (power)

e.g. 5**2



DATA TYPE/ VARIABLE

- Data in python is Case sensitive
- Data type do not need to assign in advance
- Example data type
 - `int`
 - `float`
 - `str: string`
 - `Bool: true/false`
- Check data type by : `type`



EXAMPLE ON DATA TYPE

```
a=5
```

```
b=6
```

```
print(type(a))    #Check data type
```

```
c=float(a+b)     #Data conversion
```

```
print(type(c) )
```



EXAMPLE 2

```
a=10+20  
print(a)  
print(type(a))  
s="Good morning"  
print(type(s))  
print(s + " " + str(a));
```



USER DEFINED FUNCTIONS

- syntax

```
def def_name:
```

```
    detail (don't forget the indent)
```

- Example

```
def square(x) :
```

```
    return x**2 #indent is required
```



LIST

- List is a class with a combination of data which can be from any data type

```
a=10
```

```
b="haha"
```

```
c=10.9
```

```
ls=[a , b,c] # , is used to separate each data type
```

```
print(ls)
```

```
print(type(ls))
```

- ls of ls can be created as a list of list
 - ls1=[ls, [a ,b]]



INDEXING

- The first index is '0', using [] for indexing
 - `print(ls[0])`
- Negative indexing is counting from the last element by -1
 - `print(ls[-1])`
- Sublisting [start(inclusive):end(exclusive)]
 - `ls[1:3]` → `ls[1]` and `ls[2]`
 - `ls[:3]` → `ls[0]` `ls[1]` `ls[2]`
 - `ls[2:]` → `ls[2]` till the end



LOOP

- Writing a loop
 - for x in range(5):
 print(x)
 - for x in range(2,5):
 print(x)
 - for x in range(0,len(ls)):
 print(x)
 - for x in ls:
 print(x)



INSTALL NEW PACKAGES

Before you can use any package use pip install

- **pip install** numpy

When you write your code

- `import numpy` → use the whole package
- `import numpy as np` → use np as your nickname
- `from math import pi` → use only **pi** from math



ARRAY

- List cannot be used for calculation

```
height=[1.56, 1.79 ,1.89]
```

```
sq_height=height**2 → INVALID
```

- Numpy array improves this feature

```
import numpy as np
```

```
np_height=np.array(height)
```

```
sq_height=np_height**2 → OK
```



ARRAY MANIPULATION

- **Comparison: (`==`, `>`, `<`, `<=`, `>=`)**

return boolean values of each element

```
x=np.array([1, 2, -1])
```

```
x>0 returns array{True, True, False}
```

- **Sublist from array**

```
y=x[x>10]
```



TWO-D ARRAY

```
arr=numpy.array([[1,2,3],[4,5,6]])
```

→ **Row Referencing [row]**

```
arr[0] is array([1,2,3])
```

→ **Single value Reference**

→ **[row][col] or [row,col]**

```
arr[0,1] and arr[0][1] are 2
```



NUMPY FUNCTIONS

- mean

- `np.mean(arr)` returns 3.5

- median

- `np.median(arr)` returns 3.5

- var

- `np.var(arr)` returns 2.91666



DEALING WITH ARRAY/MATRIX

Reshape

- `new_arr=arr.reshape(3,2)`

Transpose(.T)

- `new_arr=arr.T`

Flatten

- `new_arr=arr.flatten()`



CALCULATION

```
A=np.array([[1,1],[1,2],[1,3]])
```

```
B=np.array([[2,1],[2,2],[2,3]])
```

- **Add/subtract**

```
C=A+B
```

- **Dot Products**

```
np.dot(A,B.T) #Python 3.5 +can use A @B
```

- **Inverse**

```
matrix=np.array([[1,4],[2,5]])
```

```
D=np.linalg.inv(matrix)
```



RANDOM

- **Set Seed**

```
np.random.seed(0)
```

- **Uniform random over [0,1)**

```
np.random.random(3)
```

- **Normal distribution random**

```
np.random.normal(loc=0, scale=3, size=None)
```

- **Random integer within a specific range and**

```
np.random.randint(1, 10, 3) # random three numbers between 1-10
```



LOADING DATA

- From online dataset

```
from sklearn import datasets  
x=dataset.load_[name_ofdataset]
```

- From csv

- From excel

- Use Pandas to help



PANDAS PACKAGE

- PANDAS allows to extend the ability of array
 - Series for 1 dimensional data
 - Dataframe for 2 dimensional data
 - Panel for 3 dimensional data
- Label is the key part for Pandas for data accessing



LOADING CSV

- **IMPORT FROM CSV** import panda as pd

```
df = pd.read_csv('foo.csv', index_col=0, header=0, sep=':')
```

- 'foo.csv' is the file you to be imported
- index_col is the column used as index
- header indicates how many heard line before data
- sep is the separator, default is ','

Or

```
df = pd.read_csv(url); #specified url
```



COLAB COMMANDS

- `df1 = pd.read_csv(url)`
- `from google.colab import files`
`uploaded = files.upload()`
- `from google.colab import drive`
`drive.mount('/content/drive')`
`path = "my drive"`
`df_bonus = pd.read_csv(path)`

<https://towardsdatascience.com/3-ways-to-load-csv-files-into-colab-7c14fcbdc92>



LOADING EXCEL

- IMPORT FROM CSV or Excel

```
import pandas as pd
```

```
df = pd.read_excel('foo.xls',sheetname=0,header=1)
```

- **Sheetname** specifies the sheet for multiple sheets

```
sheetname=[0,1,2,"new sheet"]
```

- **header** indicates how many header line before data



LOADING JSON

- IMPORT FROM CSV or Excel

```
import pandas as pd
```

```
df = pd.read_jsonv(url, orient='column')
```

https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.io.json.json_normalize.html



DATAFRAME

```
data=np.array([[ ' ', 'C1','C2'],
               ['R1',1,2],
               ['R2',3,4],])
dataframe=pd.DataFrame(data=data[1:,1:],
                       index=data[1:,0],
                       columns=data[0,1:])

print(dataframe[0:2])
print(dataframe['C1'])
print(dataframe.shape)
dataframe.describe()
```



ACCESSING DATAFRAME

for dataframe

	C1	C2
R1	1	2
R2	3	4

TO ACCESS 1

- `df.iloc[0][0]`
- `df.loc[0]['C1']`
- `df.at[0,'C1']`
- `df.iat[0,0]`

To access R1

- `df.iloc[0]`

To access C1

- `df.loc[:, 'C1']`



MANAGING DATAFRAME

#Adding a new column

- `df['C3']`

#Adding a new column with value

- `df.loc[:, 'C3'] = pd.Series([5, 6], index=df.index)`

#Deleting Column

- `df.drop('C1', axis=1)`

#Deleting Row

`df.drop('R1', axis=0)`



REPLACE/ RENAME

- **Replace**

```
dataframe['Sex'].replace("female", "Woman")
```

```
dataframe['Sex'].replace(["female", "male"], ["Woman", "Man"])
```

- **Rename**

```
dataframe.rename(columns={'Sex': 'Gender'}).head(2)
```



TASK 1

- Import 'iris' data (.csv format)
- 1. Display the last 10 elements
- 2. Copy the second column(width) to a list
- 3. Display the first 10 elements
- 4. Create a new column called Species_label
- 5. change name in Species to 0,1,2 in Species_label



CONDITION

- `df['Species']== 'Iris-setosa'`
- `df[(df['Species']== 'Iris-setosa') & (df['Id']>=5 )].head(5)`

Question??

- How to replace **new col** with
Iris-setosa → 0
Iris-virginica --> 2



STATISTICAL VALUE

- `print('Maximum: ', dataframe['Age'].max())`
- `print('Minimum: ', dataframe['Age'].min())`
- `print('Mean: ', dataframe['Age'].mean())`
- `print('Sum: ', dataframe['Age'].sum())`

Group by

`dataframe.groupby('Sex').mean()`

Group by TIMEframe



RESAMPLE TIME DATA

```
Import pandas as pd
Import numpy as np
time_index=pd.date_range('06/06/2017',periods=100000,freq='30S')
df=pd.DataFrame(index=time_index)
df['Sale_Amount']=np.random.randint(1,10,100000)
df.resample('W').sum()
```



DROP DUPLICATE DATA

```
Dataframe.drop_duplicates()
```

You can check by

```
print(len(dataframe))
```

```
print(len(dataframe.drop_duplicates()))
```



LOOPS

```
for name in dataframe['Name'][0:5]:  
    print(name)
```



FUNCTION OVER ALL ELEMENTS

```
def uppercase(x):  
    return x.upper()  
  
dataframe['Name'].apply(uppercase)[0:2]
```



CONCAT AND MERGE

■ Concatenation

- Combine data frame using the same row
- `pd.concat([dataframe_a, dataframe_b], axis=0)`

■ Merge

- Merge data using the same key (inner join)
- `pd.merge (dataframe_a, dataframe_b, on='employee_id')`

Outer

```
pd.merge (dataframe_a, dataframe_b, on='employee_id',how='left')
```

Different name

- For outer join use `'left_on'` or `'right_on'`
- `pd.merge (dataframe_a, dataframe_b, left_on='employee_id',
right_on='id')`



VISUALIZATION

```
import matplotlib.pyplot as plt
```

```
plt.plot(year, pop)  
plt.show()
```



PLOTTING

Create the plot

- Line plot

```
plt.plot(x,y)
```

- Dot plot

```
plt.scatter(x,y) or plt.scatter(x,y,s= z) to specify the sizes
```

Histogram

```
plt.hist(life_exp) or plt.hist(life_exp,bin= 10)
```



SHOW / CLEAR THE PLOT

Start plotting

```
plt.show()
```

Clear the plot

```
plt.clf()
```



TASK 2

- Find mean/median/min/max/average for each column
- Find mean/median/min/max/average based on each type
- Plot (scatter) each data based on each type

